

# Bio-Inspired Locomotion for *Walter* (Walking Alternating Tripod for Evolutionary Research): CAD, Firmware, Optimization, MuJoCo Replay, and PPO

Howard Wang and Matthew Shiley

Engineering 90

Advisor(s): Allan Moser, Francisco Valero-Cuevas, Will Johnson

May 3, 2026

## Abstract

*Walter* (Walking Alternating Tripod for Evolutionary Research) is a twelve-degree-of-freedom hexapod built for this Engineering 90 capstone. The team fabricated a complete hardware platform: computer-aided-design models for the body, legs, and linkages; hobby servos; a Teensy 4.1 controller and Pololu eighteen-channel servo driver; SparkFun BMI270 inertial sensing; power regulation; and three iterations of custom *Hexapod V2* printed circuit boards. On the control side, the project now includes two open-loop gait optimization methods, deterministic export of optimized gait parameters to firmware, deterministic replay of both optimized gaits in the MuJoCo physics simulator on flat, rough, and stepped terrain, and Proximal Policy Optimization policies trained in MuJoCo with inertial-observation configurations. The best bounded local-search open-loop replay moved an average of 1.78 m forward across the three MuJoCo terrain tests, and the dual-annealing replay moved an average of 1.46 m after applying a servo-realistic frequency cap. The strongest inertial-observation policy evaluation moved an average of 0.70 m forward over three 1000-step MuJoCo episodes. The full layered adaptive firmware described in internal phase requirements, including inertial closed-loop reflexes and online adaptation on the microcontroller, was not completed on hardware; that stack remains future work. Overall, the capstone establishes a reproducible hexapod platform, two defensible optimized open-loop gait workflows, and simulation-based reinforcement-learning workflows with archived checkpoints and evaluation summaries. Materials are maintained at <https://github.com/HowardWHSrun/WALTER>. Purchased cost was US\$595.20 for electronics and US\$197.47 for printed-circuit-board orders, for a tracked total of US\$792.67; filament and fasteners are outside this sum.

# 1 Introduction

Reliable legged locomotion requires coordinating periodic limb motion with feedback under limited sensing and computation. Insects organize locomotor control in *layers* that are standard in the literature: rhythmic coordination (often modeled as central-pattern-generator-like structure), fast sensory feedback, and slower parameter adjustment [1, 2]. This project uses that organization as bio-inspired control motivation, not as a claim that every layer is already running on hardware.

Framing of contributions. The strongest completed work is platform and methods: a reproducible hexapod with computer-aided design (CAD) documentation and custom printed circuit boards (PCBs); two open-loop gait optimization methods that export parameters to firmware; deterministic MuJoCo replay of those optimized gaits; and a Proximal Policy Optimization (PPO) training and evaluation workflow with inertial-observation runs, stored checkpoints, videos, and configs. A layered adaptive architecture using inertial measurement unit (IMU) fusion, reflexes, and slow adaptation is specified in design documents and informs observation choices in simulation, but it is not fully implemented on the physical hexapod. The report therefore emphasizes what was built and verified, and treats full adaptive locomotion on the robot as future engineering, not an accomplishment of this submission.

Bio-inspired elements (precise). The alternating tripod is a biologically motivated low-dimensional gait prior: two stance groups alternate before any learning is applied. IMU-based body-state cues (roll, pitch, rates) match how many legged systems use gravito-inertial information for posture [3, 4] and align the simulation observation stack with how *Walter* is instrumented, which matters for eventual sim-to-real. Reinforcement learning in MuJoCo explores dynamics and reward shaping under that prior; it does not, by itself, close the hardware adaptive loop. Surrogate optimization of a compact sinusoidal law isolates an interpretable rhythm before sensor-heavy layers are added—consistent with separating pattern generation from feedback in many biological models [2], but realized here only through open-loop firmware.

Platform. The physical system is *Walter*, a twelve-degree-of-freedom (DOF) hexapod with 3D-printed structure, servos, a Teensy microcontroller unit (MCU), Pololu servo driver, IMU, and *Hexapod V2* PCBs (Sec. 6.5–6.6).

Code and report organization. Source code and this document are maintained at <https://github.com/HowardWHSrun/WALTER>. Sec. 2 states the optimization objective and the PPO machinery used in software. Sec. 3–5 and Table 1 summarize three tracks: (1) open-loop optimization, firmware export, and MuJoCo replay using limited-memory Broyden–Fletcher–Goldfarb–Shanno with box constraints (L-BFGS-B) and dual annealing (completed); (2) PPO locomotion in MuJoCo, including IMU-only and IMU-plus-phase observation modes (completed as a training/evaluation stack; limited hardware deployment); (3) vision-augmented distance objectives (future work).

## 2 Mathematical background: optimization and reinforcement learning

Terminology. In this report, a *surrogate* is a deliberately simplified model used for fast search. It is not the full robot and not the MuJoCo simulator: it is a reduced-order planar dynamics model that approximates body translation, body attitude, ground contact, and actuator-load proxies well enough to compare many candidate gait parameters quickly.

### 2.1 Reduced-order gait optimization (Track 1)

#### 2.1.1 Decision variables and bounds

The optimization problem is to find a feasible gait parameter vector  $\phi$  that minimizes the scalar cost in Eq. (5), subject to the box constraints listed below. The L-BFGS-B implementation acts on a vector  $\phi \in \mathbb{R}^{25}$  matching `hexapod_no_imu_optimization.py`. Indices are concatenated as

$$\phi = (\theta_{\text{abd,rest}}^\top, \theta_{\text{flx,rest}}^\top, \delta_{\text{abd}}^\top, \delta_{\text{flx}}^\top, f)^\top, \quad (1)$$

where each of  $\theta_{\text{abd,rest}}, \theta_{\text{flx,rest}}, \delta_{\text{abd}}, \delta_{\text{flx}} \in \mathbb{R}^6$  are in *degrees* in the code (converted to radians internally), indexed by leg order front-left (FL), front-right (FR), middle-left (ML), middle-right (MR), rear-left (RL), rear-right (RR), and  $f \in \mathbb{R}_+$  is the common gait frequency in hertz. Box constraints are implemented as `scipy.optimize.Bounds`: for example  $f \in [0.4, 4.0]$  Hz; resting abduction per joint in  $[-25^\circ, 25^\circ]$ ; resting flexion in  $[10^\circ, 65^\circ]$ ; phase shifts in  $[-120^\circ, 120^\circ]$ . These define a feasible orthotope  $\phi_{\min} \leq \phi \leq \phi_{\max}$ .

#### 2.1.2 Open-loop joint commands (sinusoidal law)

Let  $\omega = 2\pi f$ . For leg  $\ell \in \{1, \dots, 6\}$  with fixed tripod offset  $\psi_\ell \in \{0, \pi\}$  (alternating tripod), time  $t \geq 0$ , and fixed amplitude tables  $\bar{a}_{\text{abd},\ell}, \bar{a}_{\text{flx},\ell}$  (defaults in code, not decision variables), the commanded abduction and flexion angles are

$$\phi_{\text{abd},\ell}(t) = \theta_{\text{abd,rest},\ell} + \bar{a}_{\text{abd},\ell} \sin(\omega t + \psi_\ell + \delta_{\text{abd},\ell}), \quad (2)$$

$$\phi_{\text{flx},\ell}(t) = \theta_{\text{flx,rest},\ell} + \bar{a}_{\text{flx},\ell} \sin(\omega t + \psi_\ell + \delta_{\text{flx},\ell}) - \Delta_{\text{lift},\ell}(t), \quad (3)$$

where  $\Delta_{\text{lift},\ell}$  implements swing-phase foot clearance (subtracting flexion when the abduction sine is positive), identical in structure to the firmware `LIFT_SWING_DEG_PEAK` term. Velocities  $\dot{\phi}_{\text{abd},\ell}, \dot{\phi}_{\text{flx},\ell}$  are obtained by differentiation and feed the planar surrogate dynamics.

### 2.1.3 Surrogate planar dynamics and simulation

The reduced model integrates an eight-dimensional state

$$\mathbf{x}(t) = (x, \dot{x}, z, \dot{z}, \phi_{\text{roll}}, \dot{\phi}_{\text{roll}}, \phi_{\text{pitch}}, \dot{\phi}_{\text{pitch}})^\top, \quad (4)$$

with  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \phi)$  assembled from lumped masses, soft ground contact, leg thrust and support forces derived from  $\phi_{\text{abd},\ell}$ ,  $\phi_{\text{flx},\ell}$  and contact weights, and linear damping on body motion and attitude rates (`body_ode` in the script). For fixed  $\phi$ , the trajectory is obtained by `solve_ivp` (Runge–Kutta) over  $t \in [0, T]$  with  $T$  the simulation horizon (`sim_duration_s`), sampled at  $N$  time points (`sample_count`).

### 2.1.4 Scalar objective (explicit weighted sum)

Let a subscript “ss” denote statistics evaluated on a *steady-state* suffix of the horizon (the code uses the last 60% of samples for oscillation metrics and mean speed). Write  $\bar{v}_x$  for mean forward velocity  $\mathbb{E}_{\text{ss}}[\dot{x}]$ , and root-mean-square (RMS) values  $\text{RMS}_{\text{ss}}[\phi_{\text{roll}}]$ ,  $\text{RMS}_{\text{ss}}[\phi_{\text{pitch}}]$ ,  $\text{RMS}_{\text{ss}}[z]$ . Let  $q_{\text{servo}} \geq 0$  be a slack on peak torque proxy exceeding a threshold (0.85 of stall in the implementation), and  $\sigma_{\text{contact}}$  the time mean of the per-timestep standard deviation of contact weights across legs. With nonnegative weights  $w$ , matching `OptimizationConfig`, the implemented cost is

$$\begin{aligned} \mathcal{J}(\phi) = & -w_{\text{fwd}} \bar{v}_x + w_{\text{back}} [\max(0, -\bar{v}_x)]^2 \\ & + w_{\text{rp}} (\text{RMS}_{\text{ss}}[\phi_{\text{roll}}]^2 + \text{RMS}_{\text{ss}}[\phi_{\text{pitch}}]^2) + w_z \text{RMS}_{\text{ss}}[z]^2 \\ & + w_{\text{servo}} q_{\text{servo}}^2 + w_{\text{contact}} \sigma_{\text{contact}}^2 + w_{\text{reg}} \|\phi - \phi_0\|_{\text{scaled}}^2, \end{aligned} \quad (5)$$

where  $\phi_0$  is the default parameter vector and  $\|\cdot\|_{\text{scaled}}$  averages coordinatewise squared normalized deviation from the box midpoint (Tikhonov-style regularization keeping  $\phi$  in a “reasonable” neighborhood). Optional terms penalize shortfall below a target mean speed if `target_forward_speed_mps`  $> 0$ . Infeasible simulations return a large penalty ( $10^6$ ). Design intent: the weighted sum prioritizes forward progress while penalizing backward drift, attitude and vertical oscillation, actuator overload proxies, uneven leg loading, and deviation from nominal  $\phi_0$ —a deliberate trade that can admit energetic gaits with visible body motion in the surrogate.

### 2.1.5 Optimization algorithms and restarts

The problem solved in software is

$$\min_{\phi} \mathcal{J}(\phi) \quad \text{subject to} \quad \phi_{\text{min}} \leq \phi \leq \phi_{\text{max}}. \quad (6)$$

L-BFGS-B (`scipy.optimize.minimize`, method L-BFGS-B) approximates curvature of  $\mathcal{J}$  with a limited-memory inverse Hessian and projects each iterate onto the box. Each evaluation of  $\mathcal{J}$  requires one numerical integration of the surrogate. The outer loop runs  $R$  random restarts (**restarts**): one start at  $\phi_0$  and  $R - 1$  uniform samples in the box; the best  $\phi^*$  by lowest  $\mathcal{J}$  is retained. Limits `maxiter`, `maxfun`, and `ftol` cap work per restart.

Dual annealing was added as a second optimizer for the same open-loop gait family. It is a global stochastic method: early in the temperature schedule it can accept worse intermediate solutions, which helps it leave local minima, and then it refines promising regions. In this project dual annealing uses the same speed-versus-stability cost family but searches a slightly richer gait-shape parameterization, including amplitudes and lift magnitude. The resulting gait was more aggressive in the reduced model than the L-BFGS-B multistart result, so MuJoCo replay uses a servo-realistic frequency cap when evaluating it on terrain.

The code path from optimization to firmware is deterministic:  $\phi^*$  is serialized to `no_imu_optimization_summary.json`; the sync script emits `optimized_gait_params.h`; and the open-loop sine sketch evaluates the same `leg_commands()` map, applies bench degree-to-pulse gains and clamps, and streams servo targets. Thus optimization-to-code is *algebraic parity*, not a learned policy. Both optimized open-loop controllers are also replayed directly in MuJoCo, which tests whether the optimized joint schedules remain upright and move forward in the higher-fidelity contact model before hardware trials.

## 2.2 Policy optimization in MuJoCo (PPO with GAE)

Legged reinforcement learning is cast as a Markov decision process (MDP) with MuJoCo supplying transitions; we use actor-critic PPO as implemented in Stable-Baselines3 [5]. The state can include proprioception, simulated base motion, or inertial channels depending on the configuration file. Generalized advantage estimation (GAE) [6] builds advantages from temporal-difference (TD) residuals  $\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$ :

$$\hat{A}_t = \sum_{\ell=0}^{\infty} (\gamma\lambda)^\ell \delta_{t+\ell}, \quad (7)$$

with  $\lambda$  and discount  $\gamma$  set in the configuration file. PPO [6] stabilizes updates via a clipped ratio between the new and behavior policy. Define the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (8)$$

For continuous actions,  $\pi_\theta$  is typically Gaussian;  $r_t$  is the ratio of densities. The clipped surrogate objective for a batch of timesteps is

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[ \min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \quad (9)$$

with clip range  $\epsilon \in (0, 1)$  (often 0.2). Large policy steps that inflate  $r_t$  when  $\hat{A}_t > 0$  (or shrink  $r_t$  when  $\hat{A}_t < 0$ ) are clipped so the effective objective stops increasing, reducing catastrophic updates common in vanilla policy gradients.

Let  $\hat{R}_t$  denote a target return for the critic (e.g.,  $n$ -step return or GAE-backed value target). With coefficients  $c_v, c_e > 0$ , the composite loss maximized in practice is the negative of

$$\mathcal{L}_{\text{total}}(\theta, \psi) = -\mathcal{L}^{\text{CLIP}}(\theta) + c_v \mathbb{E}_t [(V_\psi(s_t) - \hat{R}_t)^2] - c_e \mathbb{E}_t [H(\pi_\theta(\cdot | s_t))], \quad (10)$$

where  $H$  is differential entropy for Gaussian policies. Minimizing  $\mathcal{L}_{\text{total}}$  corresponds to ascending  $\mathcal{L}^{\text{CLIP}}$  while fitting the critic and discouraging collapse of exploratory variance. Updates use minibatches over several epochs on each rollout buffer; hyperparameters (learning rate, rollout length `n_steps`, batch size,  $\gamma$ ,  $\lambda$ ,  $\epsilon$ ,  $c_v$ ,  $c_e$ ) are set in configuration files under `hexapod_training_new/configs/`.

### 3 Track I: Open-loop optimization, MuJoCo replay, and firmware export

This track implements Sec. 2.1: the  $\phi$  parameterization, sinusoidal `leg_commands()`, the eight-state surrogate, cost (5), box-constrained L-BFGS-B with restarts, dual annealing as a second optimizer, and a deterministic path from optimized parameters to C headers and `hexapod_openloop_sine`. The workflow isolates a tunable open-loop rhythm before IMU-closed-loop layers are required.

What we shipped. Optimizer scripts, L-BFGS-B and dual-annealing JavaScript Object Notation (JSON) summary files, JSON-to-header sync, an open-loop Arduino sketch with exported gains, and deterministic MuJoCo terrain replays for both optimized gaits. Quantitative surrogate and replay outputs appear in Tables 6–8; the high-fidelity MuJoCo asset used for replay and learning appears in Fig. 4.

### 4 Track II: PPO locomotion with inertial observations in MuJoCo

The policy  $\pi_\theta(a|s)$  conditions on proprioception (joint positions and velocities), simulator base kinematics, and—in IMU-focused configuration files—accelerometer, gyroscope, and orientation-derived channels so roll, pitch, and body rates align with how *Walter* is sensed on hardware. Tripod structure enters through phase features, structured policy heads, or reward shaping, depending on

the experiment. Training uses Eqs. (7)–(9) on MuJoCo rollouts.

Scope. The team built, trained, and evaluated policies in simulation, archived checkpoints, and wrote short evaluation summaries for representative runs (Table 9). Recent inertial-observation runs include `imu6_001`, `imu_bno10_001`, and `imu_bno10_natural_001`, each trained for about 303k MuJoCo timesteps. Deploying a trained policy as the primary closed-loop driver on the physical robot—with the same observation latency and actuation path as firmware—was out of scope for verification in this report.

## 5 Track III: Vision-augmented distance objectives (future)

Future work. Augment  $s$  with visual odometry or pose from an overhead or onboard camera so rewards can emphasize distance traveled with less privileged state. Let  $p_t \in \mathbb{R}^2$  be horizontal pose from visual integration or fiducials, and consider forward displacement  $x_t - x_0$ . A natural reward increment is

$$r_t = \alpha (x_{t+1} - x_t) - \beta \|\omega_t\|^2 - \rho \text{penalty}_{\text{fall}} - \kappa \text{TV}(a)_t, \quad (11)$$

with IMU rates  $\omega_t$  penalizing harsh attitude changes, optional terms for staying upright, and  $\text{TV}(a)_t$  denoting total variation in the action sequence. The observation becomes  $s_t = [s_t^{\text{IMU/proprio}}, f_t]$  where  $f_t$  is a vector of visual features or estimated pose; PPO then applies the same surrogate (9) with expanded input dimension.

Open challenges include sim-to-real on vision, defining rewards from estimated versus ground-truth motion, and keeping a tripod prior while learning residuals.

Table 1: Summary of the three development tracks for *Walter*.

| Track | Technical core                                                                                                                                | Outcome in this capstone                                                                                                                            |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| I     | Surrogate $\mathcal{J}(\phi)$ (5); L-BFGS-B and dual annealing; MuJoCo open-loop terrain replay; JSON $\rightarrow$ .h $\rightarrow$ firmware | Completed: two optimized gaits exported/evaluated; firmware path demonstrated                                                                       |
| II    | PPO (9); GAE (7); MuJoCo env; IMU-only and IMU-plus-phase modes in config                                                                     | Completed in simulation: training/eval pipeline, IMU-policy checkpoints, videos, and summaries; hardware policy deployment not fully exercised here |
| III   | Visual or pose features $f_t$ ; distance-centric rewards                                                                                      | Not started: specification and literature basis only                                                                                                |

## 6 Technical Discussion

### 6.1 Biological and control context

Decentralized feedback atop rhythmic coordination remains a standard idiom for insect locomotion [1, 2]. Attitude filters for accelerometer-gyro fusion [3, 7] and posture control on terrain [4] inform the IMU observation choices in simulation. Disturbance-triggered transitions [8] motivate reflex layers in the design spec, not a completed hardware demo in this capstone.

### 6.2 Planned adaptive layer (design specification)

Internal architecture notes (`docs/ARCHITECTURE.md`) describe a future two-timescale stack: a fast tick ( $\sim 10\text{--}20\text{ ms}$ ) for IMU read, attitude fusion, stability score  $S$ , reflex adjustments, gait interpolation, and servo commands; and a slow tick (per gait cycle) for windowed costs and optional parameter adaptation. That design informed IMU-inclusive observation modes in simulation, and PPO policies using those observation modes were trained in MuJoCo; the same closed-loop stack is not claimed as deployed firmware here. The complementary blend

$$\alpha \in (0, 1), \quad \theta_{\text{roll}} \leftarrow \alpha(\theta_{\text{roll}} + \omega_x \Delta t) + (1 - \alpha) \theta_{\text{roll, accel}}, \quad (12)$$

(and analogously for pitch) is the nominal attitude update.

### 6.3 Open-loop sinusoidal gait (implemented)

For the no-IMU milestone, each leg follows resting abduction and flexion angles plus sinusoidal variation at common frequency  $f$ , with tripod group B offset by  $\pi$  from group A. This is the law optimized in Sec. 2.1 and delivered in Track I (Sec. 3). The L-BFGS-B workflow keeps the firmware-facing parameter vector compact, while the dual-annealing workflow searches a more aggressive gait shape and then constrains replay frequency when moved to MuJoCo.

### 6.4 MuJoCo and reinforcement learning (simulation)

High-fidelity testing uses Gymnasium + MuJoCo [9] in two ways: deterministic replay of optimized open-loop gaits, and PPO [6] training as implemented in Stable-Baselines3 [5]. For PPO, the mathematical objective is Eq. (9) with advantages (7). Tracks II–III (Sec. 4–5) specialize the observation and reward; configuration files set  $\gamma$ ,  $\lambda$ , learning rate, network widths, proportional-derivative gains, and terrain. A representative static render of the simulation model appears in Fig. 4 (Sec. 9.3).



## 6.5 Mechanical design: CAD and 3D-printed parts

Walter’s chassis and legs are built from parts designed in CAD and produced by additive manufacturing (3D printing): a central body, six leg assemblies, and two-degree-of-freedom linkages per leg (abduction and flexion axes) that mate to the servo output splines. The design prioritizes a repeatable assembly for twelve actuated joints, clearance for wiring, and attachment of the electronics tray and battery. Figure 1 shows an overall render; Figure 2 shows the body, leg, and linkage components used in the assembly. Fasteners and servo hardware complete the mechanical stack; material usage (filament mass) and printed-part cost are not included in the dollar totals in Tables 3–5.

## 6.6 Electronics, Hexapod V2 PCB, and safety

The bill of materials matches purchased components: Teensy-class MCU (SparkFun 16771), Pololu 1354 eighteen-channel driver, buck converter, BMI270 via Qwiic cabling, dual-cell lithium-polymer battery and charger, terminals, and servos. Custom *Hexapod V2* PCBs (revision v5 dated 2026-04-03 on the latest order) integrate power and signal routing for Walter’s stack; Figure 3 shows a board photo and schematic excerpt. `docs/FIRMWARE.md` records universal asynchronous receiver/transmitter (UART) communication to the servo controller, channel ordering FL/FR/ML/MR/RL/RR with abduction then flexion per leg, and pulse mapping  $u_k = \text{clamp}(c_k + d_k q_k)$  (typical bounds 600–2400  $\mu\text{s}$ ). Open-loop firmware applies degree-to-command gains and clamps before streaming compact-protocol targets; these bench gains are not outputs of the reduced-order optimizer.

# 7 Requirements and Constraints

## 7.1 Source of requirements

Engineering requirements are traced to phase exit criteria in `docs/PHASES.md` (Phases 1–4: IMU telemetry, reflexes, cautious mode and recovery, online adaptation), supplemented by safety and interface constraints in `docs/FIRMWARE.md` and the open-loop scope in `firmware/hexapod_openloop_sine/README.md`. `PHASES.md` serves as the team’s consolidated requirements-and-constraints specification for firmware behavior.

## 7.2 Summary: met vs. not met

Satisfied in this project: discrete tripod baseline firmware; open-loop optimization with JSON export and header sync; MuJoCo replay of two optimized open-loop controllers; calibration sketches; CAD-documented mechanical design (Figs. 1–2); a MuJoCo PPO workflow with configs, training

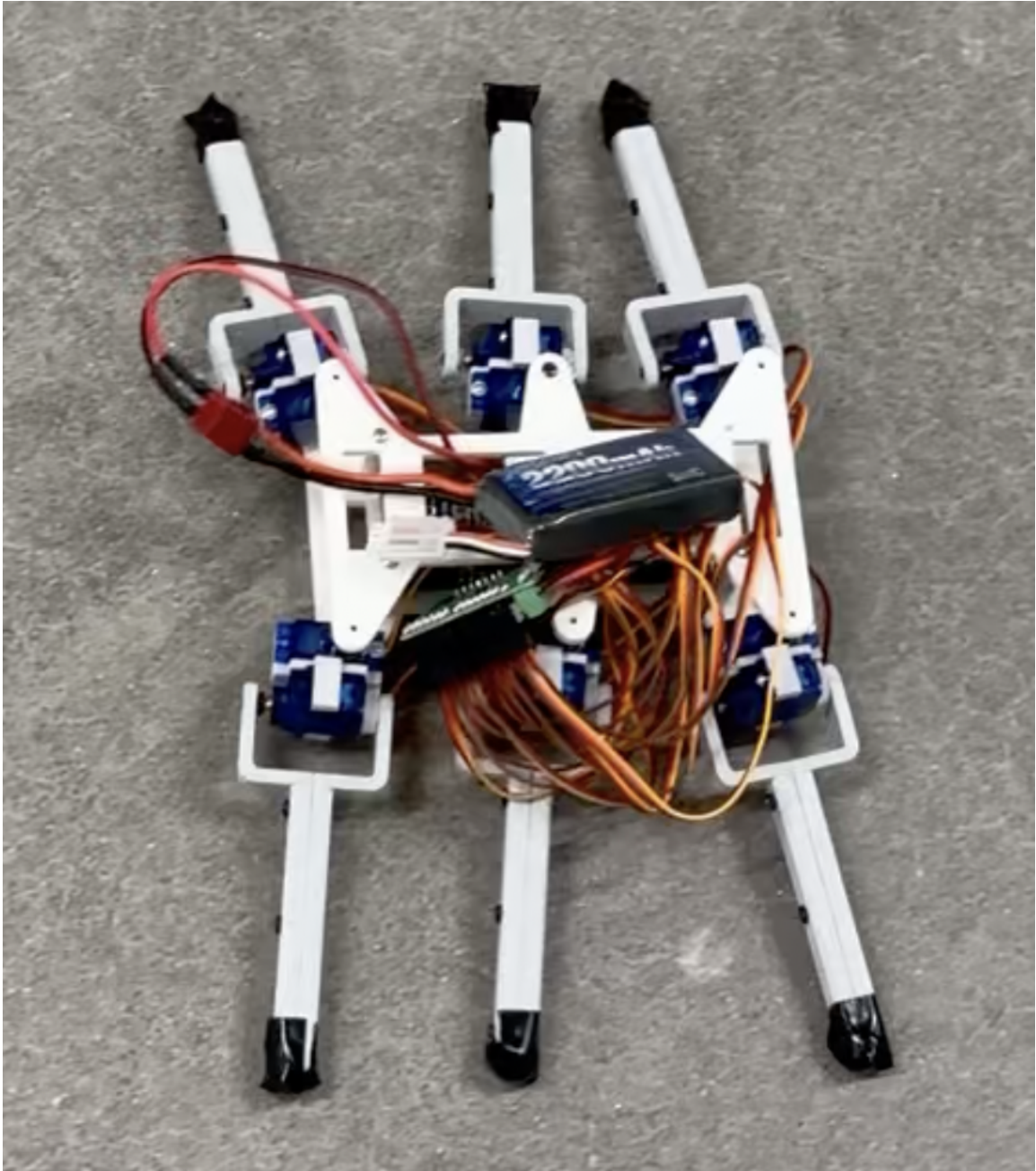


Figure 1: Walter: full-assembly render of the twelve-DOF hexapod (CAD; Walking Alternating Tripod for Evolutionary Research).

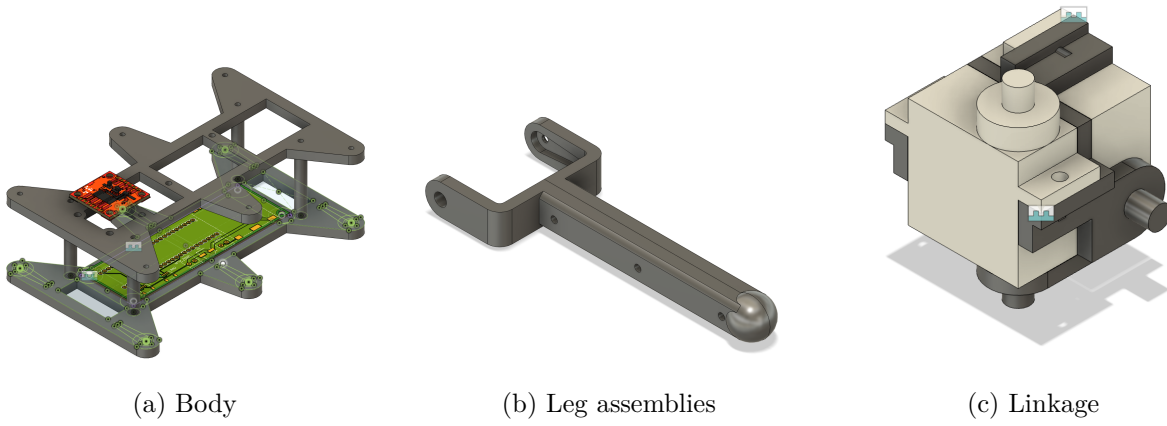


Figure 2: CAD-designed 3D-printed subassemblies for Walter: chassis, legs, and per-leg linkage.

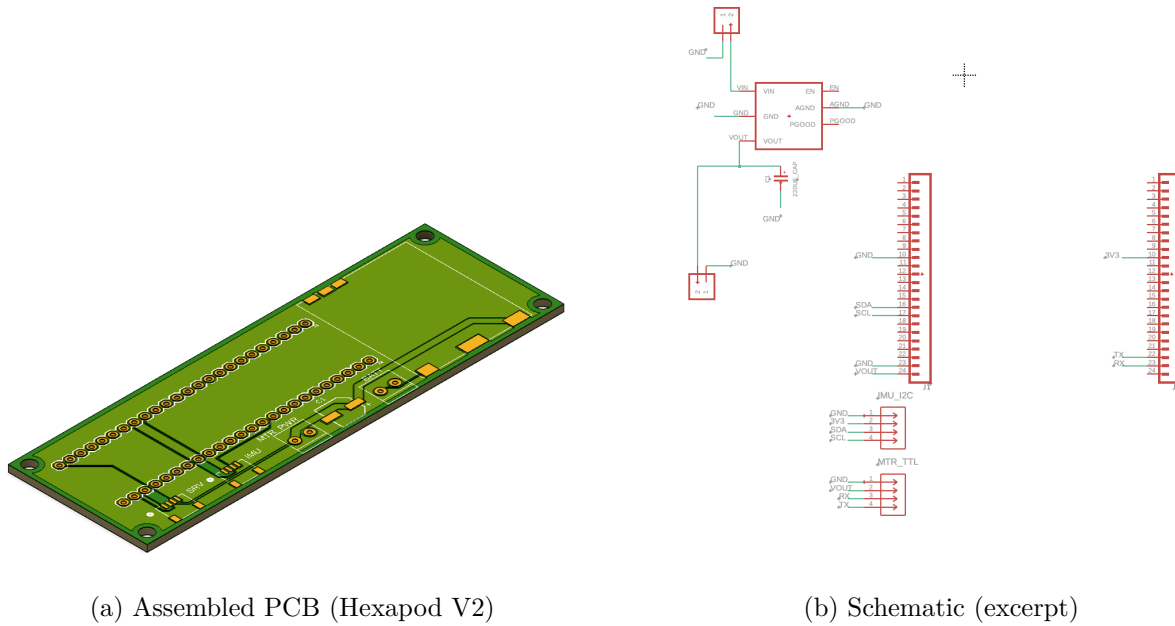


Figure 3: Custom electronics for Walter: Hexapod V2 board and schematic.

scripts, checkpoints, videos, and evaluation writeups, including inertial-observation runs. Not satisfied against the internal phased firmware spec: Phases 1–4 adaptive behaviors (telemetry-only through online adaptation) are largely not implemented on the MCU in the snapshot summarized below.

For Table 2, Universal Serial Bus (USB) is the debugging and telemetry connection, electrically erasable programmable read-only memory (EEPROM) is the microcontroller nonvolatile parameter store, and simultaneous perturbation stochastic approximation (SPSA) is the planned lightweight adaptation optimizer.

Table 2: Requirement traceability (internal PHASES.md vs. delivered work).

| ID | Requirement                                                                                               | Status            | Evidence / notes                                                                                    |
|----|-----------------------------------------------------------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------|
| R1 | Phase 1: IMU read, complementary filter, USB telemetry $\sim 50$ Hz; gait unchanged                       | Not met (in repo) | PHASES.md; folder may be missing                                                                    |
| R2 | Phase 2: postural reflexes, clamps, serial tuning, EEPROM                                                 | Not met (in repo) | ARCHITECTURE.md pseudocode                                                                          |
| R3 | Phase 3: stability score, cautious mode, push recovery, fall freeze                                       | Not met (in repo) | Documented only                                                                                     |
| R4 | Phase 4: five-parameter adaptation, cost $J$ , SPSA/bandit, EEPROM                                        | Not met (in repo) | Documented only                                                                                     |
| R5 | Hardware: MCU + eighteen-channel driver + servos + documented map + PCBs + Walter mechanical CAD/3D parts | Met               | Bill of materials, Figs. 1–3, FIRMWARE.md                                                           |
| R6 | Safety: pulse clamps; parameter slew (adaptive layer)                                                     | Partial           | Clamps in v2/openloop; slew in spec                                                                 |
| R7 | Open-loop: optimizer/firmware parity; JSON export; MuJoCo replay of optimized gait                        | Met               | L-BFGS-B and dual-annealing summaries; sync_gait_params_from_json.py; open-loop MuJoCo eval folders |
| R8 | Simulation: MuJoCo env, PPO training, IMU-observation runs, documented evals                              | Met               | hexapod_training_new/, runs/imu6_001/, runs/imu_bno10_001/, released checkpoints                    |

### 7.3 Project cost

Tables 3–4 summarize purchased electronics (three identical platforms) and fabrication costs for three PCB/stencil orders. Table 5 states the grand total for those categories only; filament, fasteners, and shop labor are outside the accounting. Servo counts in Table 3 are per platform (eight units listed per bill-of-materials line); completing twelve DOF per robot may require additional

servos beyond those lines if not already stocked.

Table 3: Electronics bill of materials, one platform (USD). The team purchased three identical sets.

| Part                      | Description                               | Vendor  | Qty | Unit  | Line   |
|---------------------------|-------------------------------------------|---------|-----|-------|--------|
| SparkFun 16771            | Microcontroller                           | DigiKey | 1   | 37.20 | 37.20  |
| Pololu 1354               | 18-channel servo driver                   | DigiKey | 1   | 46.95 | 46.95  |
| DFRobot<br>DFR1202        | Step-down buck converter                  | DFRobot | 1   | 12.90 | 12.90  |
| SparkFun<br>BMI270        | 6-DoF IMU                                 | DigiKey | 1   | 18.50 | 18.50  |
| SparkFun 11855            | 2×7.4 V lithium-polymer battery + charger | Amazon  | 1   | 27.99 | 27.99  |
| SparkFun 14417            | Female Qwiic connector                    | DigiKey | 2   | 0.60  | 1.20   |
| SparkFun 17259            | 100 mm Qwiic cable                        | DigiKey | 1   | 1.95  | 1.95   |
| SparkFun 17261            | Qwiic cable to female header              | DigiKey | 1   | 1.95  | 1.95   |
| Amphenol<br>YO0201500000G | 1×2 screw-down terminal                   | DigiKey | 2   | 1.28  | 2.56   |
| DFRobot<br>SER0039        | Servo motor                               | DigiKey | 8   | 5.90  | 47.20  |
| Subtotal                  | one platform                              |         |     |       | 198.40 |
| Electronics total         | three platforms (×3)                      |         |     |       | 595.20 |

Table 4: PCB and stencil orders, *Hexapod V2* (USD order totals including merchandise, shipping, duties/taxes, and sales tax where charged by the vendor).

| Order ID          | Description / notes                                | Status           | Total  |
|-------------------|----------------------------------------------------|------------------|--------|
| Y4-11911448A      | PCB prototype (5 pcs) + SMT stencil; v5_2026-04-03 | Awaiting payment | 74.24  |
| W2026030804205410 | PCB + stencil shipment                             | Shipped          | 68.26  |
| W2026021709554129 | PCB + stencil (earlier order)                      | Shipped          | 54.97  |
| PCB subtotal      |                                                    |                  | 197.47 |

Table 5: Grand total project cost (USD) for reported purchases and fabrication.

| Category                                    | Amount |
|---------------------------------------------|--------|
| Electronics (three platforms)               | 595.20 |
| PCB fabrication and stencils (three orders) | 197.47 |
| Grand total                                 | 792.67 |

## 8 Methods

### 8.1 Mechanical assembly (*Walter*)

Printed body and leg components are fastened per the CAD design; each leg receives two servos driving abduction and flexion through the linkage. The custom PCB mounts to the chassis and

connects to the Pololu driver and Teensy; power and ground respect the buck converter and screw terminals from the bill of materials. Bench alignment uses `set_zero` and side-only test sketches before closed-loop or open-loop gait tests.

## 8.2 Baseline discrete gait

`firmware/hexapod_walking_v2/hexapod_walking_v2.ino` implements a deterministic tripod state machine with smoothed keyframes (`moveSmoothTo`).

## 8.3 Reduced-order simulation optimization

`hexapod_brain_roadmapcopy/ode_optimization_no_imu/hexapod_no_imu_optimization.py` integrates the surrogate model with `scipy.integrate.solve_ivp`, evaluates the objective on a steady-state window, and calls `scipy.optimize.minimize` (L-BFGS-B). A second summary, `no_imu_optimization_da_summary.json`, records the dual-annealing result. Outputs include JSON summaries of optimized frequency, angles, phase shifts, lift, objective breakdowns, and optional comma-separated-value files and plots.

## 8.4 Parameter sync and open-loop firmware

From `firmware/hexapod_openloop_sine/`, run the sync script to regenerate `optimized_gait_params.h`. The sketch evaluates the sinusoidal law at `INTERP_DELAY_MS` (default 10 ms), applies startup frequency ramping, maps degrees to integer commands, clamps, and sends Pololu compact-protocol targets on `Serial1`.

## 8.5 MuJoCo replay of optimized open-loop gaits

`hexapod_training_new` replays the optimized open-loop controllers in MuJoCo without training a policy. The replay maps firmware leg order front-left, front-right, middle-left, middle-right, rear-left, rear-right to the MuJoCo model order, applies the required abduction and flexion sign conventions, initializes the robot in the first gait pose, rate-limits target changes, and evaluates one 1000-step episode each on flat, rough, and stepped terrain. The L-BFGS-B gait is replayed at the firmware-capped 1.0 Hz; the dual-annealing gait also uses a 1.0 Hz cap because its raw 3.638 Hz reduced-model optimum was too aggressive for terrain replay.

## 8.6 Bench testing and calibration

`firmware/set_zero/` and side-only test sketches support calibration. The open-loop README documents sign and scale tuning when feet drag or the robot shuffles backward.

## 8.7 MuJoCo PPO training and evaluation

The project produced a reusable training workflow: `train.py` reads configuration files from `configs/`, writes summaries, checkpoints, and logs under `runs/<run_name>/`, and pairs with `evaluate.py` using the same configuration. Inertial-observation runs use modes such as `imu6` and `imu_bno10`, adding simulated accelerometer, gyroscope, and orientation-derived channels with configured noise. Released material includes zipped policies, per-run evaluation folders with summaries, plots, and videos, and metadata for reproducibility and reporting.

## 9 Results

### 9.1 Reduced-order optimization (surrogate model)

Table 6 summarizes the two reduced-order optimizer outputs used later in MuJoCo replay. These numbers describe the *planar surrogate* only; they are not a guarantee of on-robot speed or stability.

Table 6: Optimizer-reported source metrics before MuJoCo replay.

| Optimizer           | $f$ (Hz) | Mean $v_x$ (m/s) | Travel (m) | Roll RMS (rad) | Lift (deg) | Cost    |
|---------------------|----------|------------------|------------|----------------|------------|---------|
| L-BFGS-B multistart | 1.800    | 0.234            | 1.866      | 0.539          | 12.0       | −226.24 |
| Dual annealing      | 3.638    | 0.655            | 5.192      | 0.966          | 9.12       | −630.90 |

Reading Table 6. Both optimizers found forward-moving source gaits in the reduced model. Dual annealing achieved a much higher surrogate speed and lower source cost, but it did so partly through high frequency and larger body motion. That is why MuJoCo replay, not the surrogate score alone, is used to judge whether the gait geometry transfers to a richer contact model. Pitch RMS remains near zero in both source files because the reduced planar model is symmetric; it should not be interpreted as a hardware pitch-stability claim.

Table 7 lists the L-BFGS-B  $\phi^*$  exported to `optimized_gait_params.h`. Flexion phase shifts sit at the lower bound ( $-90^\circ$ ): the optimizer traded stride phasing aggressively for the surrogate objective; bench tuning may move off that bound. Rest angles cluster in physically plausible ranges; frequency near 1.8 Hz matches a moderate stride rate for small hexapods in simulation-like settings. The dual-annealing gait is retained in its own JSON summary and replayed in MuJoCo, but the firmware header path currently uses the L-BFGS-B compact parameter set.

### 9.2 MuJoCo replay of optimized open-loop controllers

Table 8 summarizes deterministic MuJoCo terrain tests for the two optimized open-loop controllers. Each row averages one 1000-step episode each on flat, rough, and stepped terrain. These are not learned policies; they are direct replays of optimized gait schedules after correcting leg-order and



Table 7: Optimized decision variables  $\phi^*$  from L-BFGS-B (same JSON as Table 6). Angles in degrees; leg order is front-left, front-right, middle-left, middle-right, rear-left, rear-right.

| Quantity                                         | Value                                            |
|--------------------------------------------------|--------------------------------------------------|
| Gait frequency $f$                               | 1.800 Hz                                         |
| Abduction rest $\theta_{\text{abd},\text{rest}}$ | (10, 10, 0, 0, -10, -10)                         |
| Flexion rest $\theta_{\text{flx},\text{rest}}$   | (32.00, 32.00, 34.00, 34.00, 32.00, 32.00)       |
| Abduction phase $\delta_{\text{abd}}$            | $\approx \mathbf{0}$ (magnitude $< 10^{-6}$ deg) |
| Flexion phase $\delta_{\text{flx}}$              | (-90, -90, -90, -90, -90, -90)                   |
| Swing lift amplitude                             | 12.0 deg ( <code>lift_swing_deg_peak</code> )    |

joint-sign conventions.

Table 8: Optimized open-loop controllers replayed in MuJoCo terrain tests.

| Controller     | Replay $f$ (Hz) | Mean $+x$ (m) | Mean $v_x$ (m/s) | Max tilt (deg) | Notes                                                                                                             |
|----------------|-----------------|---------------|------------------|----------------|-------------------------------------------------------------------------------------------------------------------|
| L-BFGS-B sine  | 1.0             | 1.776         | 0.0844           | 5.68           | Stable across flat, rough, and stepped terrain; very small lateral drift and mean final yaw of 2.06°.             |
| Dual annealing | 1.0             | 1.459         | 0.0705           | 6.20           | More aggressive source gait; frequency cap prevents the raw 3.638 Hz optimum from becoming unrealistic in MuJoCo. |

Reading Table 8. The L-BFGS-B gait is less aggressive in the surrogate but transfers cleanly to MuJoCo terrain replay. The dual-annealing gait confirms that the second optimizer found usable gait geometry, but its source timing must be constrained for realistic replay. This is the clearest evidence that open-loop optimization is now running through the MuJoCo validation step rather than ending at a reduced-order score.

### 9.3 MuJoCo simulation (high-fidelity model)

Track I–III testing uses Gymnasium with the MuJoCo asset `hexapod_training_new/assets/hexapod.xml`: checkerboard ground, twelve torque-controlled joints, and fourth-order Runge–Kutta (RK4) integration at 0.002 s (see configs for frame skipping). Figure 4 is an offscreen MuJoCo render (software `Renderer`, no interactive viewer): the model was advanced under gravity with zero actuator commands until the body settled on the plane, then a fixed camera was used. The view documents the mesh, materials, and lighting used for open-loop replay and PPO rollouts; it is not a policy-controlled frame. Interactive inspection is available via `scripts/live_mujoco_viewer.py` with a trained checkpoint.

Figure 4 confirms the asset geometry and contact setup used for open-loop replay and learning; it is a passive settle, not evidence of policy quality. Training and evaluation videos for specific checkpoints provide the appropriate visual readout of learned behavior.





Figure 4: MuJoCo screenshot of the Walter hexapod model used for open-loop replay, PPO training, and evaluation (post-settle, zero control; offscreen render).

## 9.4 MuJoCo PPO evaluations, including IMU policies

Table 9 quotes short evaluation protocols documented under [e90\\_repo/released\\_checkpoints/evaluations/](#) and [hexapod\\_training\\_new/runs/](#). Rewards are not comparable across configs (different shaping and scaling); forward displacement and path length provide a common physical readout within each experiment family.

Table 9: Representative PPO evaluation means. Episode length is 300 steps for the released checkpoint rows and 1000 steps for the IMU rows unless noted.

| Checkpoint                                    | Task / policy                                                                                                         | Fwd. $+x$              | 2D path | Notes                                                                                                                                |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">mlp_clean_run005_best</a>         | Circle path; multilayer perceptron (MLP) $256 \times 256$ PPO; training run <a href="#">mlp_run_005</a> (301k steps). | 0.18 m                 | 0.27 m  | Mean reward $\approx 6800$ .                                                                                                         |
| <a href="#">terrain_low_shake_run001_best</a> | Flat, rough, and step terrain; MLP PPO.                                                                               | 0.33 m<br>(3 ep. mean) | —       | Flat/rough $\approx 0.47$ m forward; steps harder.                                                                                   |
| BNO10 IMU                                     | MuJoCo PPO with ten-channel inertial observation mode; three video evaluation episodes.                               | 0.70 m                 | 1.71 m  | Strongest IMU-policy row; mean reward $\approx 18,174$ ; run folder <a href="#">imu_bno10_001</a> .                                  |
| BNO10 IMU + phase                             | Inertial observation plus phase feature; smoother/natural-action tuning.                                              | 0.41 m                 | 1.46 m  | Mean reward $\approx 14,448$ ; consistent forward progress across three episodes; run folder <a href="#">imu_bno10_natural_001</a> . |
| Six-channel IMU                               | Six-channel inertial observation; final-model MuJoCo video evaluation.                                                | 0.089 m                | 0.186 m | Demonstrates the PPO-IMU stack running in MuJoCo, but with modest forward progress; run folder <a href="#">imu6_001</a> .            |

Reading Table 9. The rows are not comparable by raw reward (shaping and scaling differ). Forward displacement and path length are coarse physical readouts: the circle-path MLP run shows modest net forward motion and path length consistent with a curved task; the terrain run averages shorter forward progress when steps and roughness dominate; the inertial-observation rows show that PPO policies can be trained and evaluated in MuJoCo using simulated IMU-style inputs. The [imu\\_bno10\\_001](#) row is the strongest recent IMU result, while the [imu6\\_001](#) row is best treated as a pipeline validation. Treat simulation displacements as indicative, not predictive of hardware distance without deployment tests.

The [pd\\_velocity\\_window\\_run018\\_best](#) checkpoint is a structured central-pattern-generator plus proportional-derivative velocity-window experiment with small net displacement in its released short evaluation—useful as an archival structured-policy datapoint, not a headline locomotion result (see that folder’s `REPORT.md`).

## 10 Discussion

What this E90 demonstrates. The team built a named hexapod platform with documented mechanics, electronics, and cost; implemented an optimization-to-firmware path with two optimization methods; replayed the optimized open-loop controllers in MuJoCo; and ran a MuJoCo PPO training and evaluation workflow with archived checkpoints, videos, and IMU-observation configs. Together, that is a credible platform-and-methods outcome: reusable hardware, defensible open-loop pipelines, MuJoCo validation of optimized gait schedules, and simulation evidence for learned locomotion behaviors under stated rewards.

Why full adaptive locomotion is not claimed. Closing adaptive loops on hardware requires time for IMU bring-up, reflex tuning, safety testing, and power-aware deployment. The phased firmware requirements in `PHASES.md` were not met in the repository snapshot (Table 2); stating that once in requirements and here avoids repeating it in every technical subsection.

Technical bottlenecks. Surrogate versus MuJoCo versus hardware disagree on contact, friction, and servo dynamics; the dual-annealing source optimum illustrates this because its raw high frequency must be capped for MuJoCo terrain replay. Manual degree-to-pulse mapping separates optimized angles from reliable motion; open-loop gaits do not reject disturbances. Reinforcement-learning rewards are task-specific, so cross-comparing raw returns across configuration files is misleading—compare displacements and behaviors within a task family only, with humility about sim-to-real.

Sim claims. MuJoCo results show that optimized open-loop gait schedules and PPO policies *can* run under the project’s models and rewards. They are not automatic predictors of field performance without transfer experiments.

Credible next steps. Use the L-BFGS-B MuJoCo replay as the first hardware gait candidate; compare it against the dual-annealing geometry at conservative frequency; harden IMU read paths and logging on the Teensy; then pick one IMU-PPO checkpoint and measure a short hardware rollout with matched observation rate before adding vision or distance-only objectives.

Advice for future teams. Lock a baseline gait and logging format before layering sensors; budget bench time after every simulation milestone; track PCB revisions and BOM multiples from week one.

## 11 Conclusion

This capstone established *Walter*: a twelve-DOF hexapod with CAD/3D-printed structure, custom PCBs, and purchased electronics totaling US\$792.67 in tracked spending. The team completed open-loop gait optimization with two methods, deterministic export to firmware, and MuJoCo replay of the optimized controllers. The team also completed a MuJoCo PPO training and evaluation toolchain with stored checkpoints, representative evaluations, and recent IMU-observation policy

runs. A bio-inspired, layered adaptive control story informed design and simulation choices but was not fully realized on the MCU; that gap is documented in requirements traceability (Table 2) rather than hidden. The project provides a strong base for future adaptive and sim-to-real work; code and this report are maintained at <https://github.com/HowardWHSrun/WALTER>.

## Acknowledgements

We thank our advisor, Allan Moser, for guidance on the project. We thank Swarthmore Engineering and shop or lab staff who supported fabrication and testing. No funding sources beyond the Engineering department are declared for this accounting.

## References

- [1] Y. O. Aydin *et al.*, “Mechanosensory control of locomotion in animals and robots: Moving forward,” *Biomimetics*, vol. 8, no. 4, p. 211, 2023. PMC10445419.
- [2] G. Chen *et al.*, “Adaptive locomotion control of a hexapod robot via bio-inspired learning,” *Frontiers in Neurorobotics*, vol. 15, p. 627157, 2021.
- [3] R. G. Valenti, I. Dryanovski, and J. Xiao, “Keeping a good attitude: A quaternion-based orientation filter for IMUs and MARGs,” *Sensors*, vol. 15, no. 8, pp. 19302–19330, 2015.
- [4] L. Bai *et al.*, “Development and implementation of a new approach for posture control of a hexapod robot to walk in irregular terrains,” *Robotica*, vol. 37, no. 12, pp. 2225–2238, 2019.
- [5] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations.” *Journal of Machine Learning Research*, 2021.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [7] R. Zhang *et al.*, “Attitude estimation algorithm of portable mobile robot based on complementary filter,” *Micromachines*, vol. 12, no. 11, p. 1373, 2021.
- [8] A. Johnson, K. D. Gregg, and D. E. Koditschek, “Disturbance detection, identification, and recovery by gait transition in legged robots,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 3627–3634, 2010.
- [9] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 5026–5033, 2012.

## A Public repository

The public project URL is <https://github.com/HowardWHSrun/WALTER>. Appendix paths below refer to the local Engineering 90 workspace layout at the time of writing; the GitHub repository is intended to subsume or mirror these components for reproducibility.

## B Repository map (selected local paths)

- Adaptive firmware and docs: `adaptive_ondevice_hexapod/README.md`, `docs/DESIGN.md`, `docs/ARCHITECTURE.md`, `docs/PHASES.md`, `docs/FIRMWARE.md`
- Firmware sketches: `firmware/hexapod_walking_v2/`, `firmware/hexapod_openloop_sine/`, `firmware/set_zero/`, side-only tests
- Reduced-order optimization: `hexapod_brain_roadmapcopy/ode_optimization_no_imu/`
- Open-loop sine replay: `runs/openloop_sine_mujoco_straight_firmware_terrain_001/`
- Dual-annealing replay: `runs/openloop_da_mujoco_straight_freqcap_terrain_001/`
- MuJoCo PPO training: `hexapod_training_new/` (`train.py`, `configs/`, `runs/`)
- Recent IMU runs: `runs/imu6_001/`, `runs/imu_bno10_001/`, `runs/imu_bno10_natural_001/`
- Released checkpoints and evals: `e90_repo/released_checkpoints/`
- Presentation notes: `Presentation/E90_MidSemester_SpeakerScript.tex` and portable-document-format slides
- This folder (figures): `walter_assembly.png`, `Body.png`, `Legs.png`, `Linkage.png`, `pcb_board.png`, `pcb_schematic.png`, `mujoco_hexapod_sim.png`